

Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Estado de México, Escuela de Ingeniería y Ciencias.

Unidad de formación:

TC1002S Herramientas computacionales: el arte de la analítica

Grupo 201

Profesor: Aram Baruch González Pérez

Actividad Evaluable: Limpieza de datos

Estudiante:

A01799122 - Marco Antonio González Fernández

A01799928 - Mario Alberto Perez Barrera

Fecha de entrega:

24 de octubre de 2024

1. Introducción

Este reporte detalla el proceso de limpieza de un conjunto de datos relacionado con compradores de bicicletas. El análisis de los datos requiere una calidad óptima para garantizar que las conclusiones sean precisas y útiles. Por ello, el proceso se divide en tres etapas clave: inspección, limpieza, y análisis de datos. Cada una de estas fases fue diseñada para identificar y corregir errores, asegurar la integridad de los datos, y prepararlos para el análisis posterior.

1.1. Conjunto de Datos

El conjunto de datos incluye información demográfica y de comportamiento de compradores de bicicletas, como edad, ingresos, número de hijos, número de automóviles, estado civil, entre otros. Este archivo en formato CSV tiene 1252 filas y 13 columnas que representan tanto datos numéricos como categóricos. Su origen simulado refleja un escenario realista para el análisis de comportamiento de consumidores.

1.2. Objetivos de la Limpieza

La limpieza de datos se realizó para mejorar la fiabilidad y precisión de los análisis futuros. Los objetivos incluyen:

- Corrección de valores incorrectos o inconsistentes.
- Manejo de valores faltantes mediante imputación.
- Eliminación de duplicados y datos irrelevantes.

Este proceso asegura que las etapas posteriores, como el análisis descriptivo y predictivo, se basen en datos limpios y estructurados, lo que incrementa la validez de los resultados obtenidos.

2. Exploración Inicial

En la etapa de inspección, se revisaron las características básicas del conjunto de datos. Se utilizó una combinación de métodos exploratorios como la visualización de las primeras filas, un resumen estadístico, y la identificación de valores faltantes. Se observaron los siguientes problemas:

- Valores faltantes en variables importantes como Income (ingresos), Children (número de hijos), y Age (edad).
- Errores en tipos de datos, donde algunas variables numéricas, como Income, estaban almacenadas como texto.
- Posibles valores atípicos en la columna Age, con algunos registros fuera del rango esperado (por debajo de 0 o por encima de 100 años).

- Este diagnóstico inicial justificó la necesidad de una limpieza y transformación profunda antes del análisis.

```
In [22]: # Inspección inicial
print("Primeras filas del conjunto de datos:")
df.head()
```

Primeras filas del conjunto de datos:

	ID	Marital Status	Gender	Income	Children	Education	Occupation	Home Owner	Cars	Commute Distance	Region	Age	Purchased Bike
0	12591	Married	Female	30,000	4.0	Graduate Degree	Clerical	Yes	0.0	0-1 Miles	Europe	45.0	No
1	14058	Single	Male	70,000	0.0	Bachelors	Professional	No	1.0	5-10 Miles	Pacific	41.0	Yes
2	23962	Married	Female	10,000	0.0	Partial High School	Manual	Yes	2.0	1-2 Miles	Europe	32.0	No
3	23419	Single	Female	70,000	5.0	Bachelors	Professional	Yes	3.0	10+ Miles	Pacific	39.0	No
4	21940	Married	Male	90,000	5.0	Graduate Degree	Professional	Yes	0.0	0-1 Miles	North America	47.0	Yes

```
In [23]: #Lectura de La informacion de La base de datos
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1251 entries, 0 to 1250
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ID                    1251 non-null  int64
1   Marital Status        1244 non-null  object
2   Gender                1238 non-null  object
3   Income                1243 non-null  object
4   Children              1238 non-null  float64
5   Education             1251 non-null  object
6   Occupation            1251 non-null  object
7   Home Owner            1247 non-null  object
8   Cars                  1242 non-null  float64
9   Commute Distance      1251 non-null  object
10  Region                1251 non-null  object
11  Age                   1238 non-null  float64
12  Purchased Bike        1251 non-null  object
dtypes: float64(3), int64(1), object(9)
memory usage: 127.2+ KB
```

```
In [24]: #Resumen estadístico de el DataFrame.
df.describe()
```

	ID	Children	Cars	Age
count	1251.000000	1238.000000	1242.000000	1238.000000
mean	20030.208633	1.929725	1.479066	44.058966
std	5331.451777	1.638977	1.121885	11.271138
min	11000.000000	0.000000	0.000000	25.000000
25%	15465.000000	0.000000	1.000000	35.000000
50%	19731.000000	2.000000	1.000000	43.000000
75%	24549.000000	3.000000	2.000000	52.000000
max	29447.000000	5.000000	4.000000	89.000000

```
In [25]: # Revisión de valores faltantes
print(df.isnull().sum())
```

```
ID                0
Marital Status    7
Gender            13
Income            8
Children          13
Education         0
Occupation        0
Home Owner        4
Cars              9
Commute Distance  0
Region            0
Age              13
Purchased Bike    0
dtype: int64
```

3. Proceso de Limpieza

En la etapa de limpieza, se implementaron varias acciones para corregir los problemas identificados durante la inspección. Este proceso se centró en asegurar la coherencia y completitud de los datos.

```
In [33]: # Verificar nuevamente si existen valores faltantes
print(df.isnull().sum())
```

```
ID          0
Marital Status 0
Gender       0
Income       0
Children     0
Education    0
Occupation   0
Home Owner   0
Cars         0
Commute Distance 0
Region       0
Age          0
Purchased Bike 0
dtype: int64
```

```
In [34]: # Guardar los datos limpios en un nuevo archivo CSV
df.to_csv('datos_limpiados.csv', index=False)
```

3.1. Tratamiento de Valores Faltantes

El tratamiento de los valores faltantes fue fundamental. Se optó por rellenar los valores de las columnas numéricas como Children y Cars utilizando la mediana, ya que es menos sensible a valores extremos. En las columnas categóricas como Gender, Home Owner, Marital Status, y Income, se utilizó la moda para completar los valores faltantes, asegurando que los datos fueran lo más representativos posible.

```
In [32]: # Rellenar valores faltantes con la mediana en las columnas numéricas
df['Children'] = df['Children'].fillna(df['Children'].median())
df['Cars'] = df['Cars'].fillna(df['Cars'].median())

# Si hay valores faltantes en otras columnas categóricas, rellenarlos con el valor más frecuente
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
df['Home Owner'] = df['Home Owner'].fillna(df['Home Owner'].mode()[0])
df['Marital Status'] = df['Marital Status'].fillna(df['Marital Status'].mode()[0])
df['Income'] = df['Income'].fillna(df['Income'].mode()[0])
df['Age'] = df['Age'].fillna(df['Age'].mode()[0])
```

3.2. Corrección de Datos Erróneos

Para corregir los errores de formato, se transformaron las columnas como Income, que estaban almacenadas como texto, a un formato numérico. Además, se eliminaron filas duplicadas para evitar sesgos y se revisaron los valores en la columna Age, eliminando aquellos que no tenían sentido en un contexto demográfico.

```
In [30]: # Si hay duplicados, eliminarlos
df = df.drop_duplicates()
```

3.3. Manejo de Valores Atípicos

En la columna Age, se identificaron valores atípicos como edades por debajo de 0 y por encima de 100 años. Estos outliers fueron eliminados, ya que no tienen validez en el contexto de compradores de bicicletas. En el caso de Income, aunque se detectaron algunos valores extremos, estos no fueron eliminados, ya que representan casos plausibles de altos ingresos, que podrían ser relevantes para el análisis futuro.

```
In [27]: #Identificación de datos fuera del rango esperado
#La columna 'Age' tiene un rango de 0 a 100

print(df[df['Age'] < 0]) # Ver si hay edades negativas
print(df[df['Age'] > 100]) # Ver si hay edades inusualmente altas

Empty DataFrame
Columns: [ID, Marital Status, Gender, Income, Children, Education, Occupation, Home Owner, Cars, Commute Distance, Region, Age, Purchased Bike]
Index: []
Empty DataFrame
Columns: [ID, Marital Status, Gender, Income, Children, Education, Occupation, Home Owner, Cars, Commute Distance, Region, Age, Purchased Bike]
Index: []
```

3.4. Estandarización y Normalización

Aunque no fue necesario realizar una normalización intensiva, algunas columnas como Income y Children fueron transformadas para garantizar consistencia en sus formatos numéricos. Este paso fue esencial para evitar problemas durante el análisis estadístico.

```
In [28]: # Transformar columnas numéricas que están como 'object', eliminando comas si es necesario
df['Income'] = pd.to_numeric(df['Income'].str.replace(',', ''), errors='coerce')
```

4. Resultados

Al completar la etapa de limpieza, el conjunto de datos resultante fue significativamente más consistente y libre de errores, proporcionando una base sólida para su análisis.

```
In [33]: # Verificar nuevamente si existen valores faltantes
print(df.isnull().sum())
```

```
ID          0
Marital Status  0
Gender       0
Income       0
Children     0
Education    0
Occupation   0
Home Owner   0
Cars         0
Commute Distance  0
Region       0
Age          0
Purchased Bike  0
dtype: int64
```

```
In [34]: # Guardar los datos limpios en un nuevo archivo CSV
df.to_csv('datos_limpiados.csv', index=False)
```

4.1. Estadísticas Descriptivas

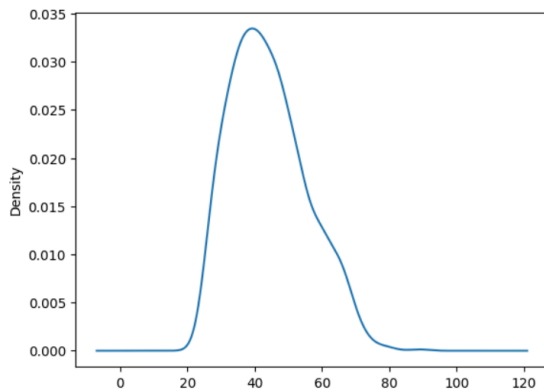
Antes de la limpieza, las estadísticas descriptivas mostraban distribuciones distorsionadas por valores faltantes y erróneos. Después de la limpieza, las estadísticas clave, como media, mediana, y cuartiles, reflejan más fielmente la realidad de los datos. Variables como Age y Income ahora tienen distribuciones más coherentes y ajustadas.

4.2. Distribución de Datos

Se generaron visualizaciones, como gráficos KDE y boxplots, para analizar la distribución de las variables clave antes y después de la limpieza. En la columna Age, los outliers fueron eliminados, resultando en una distribución más uniforme. Para Income, los boxplots mostraron la presencia de algunos valores extremos, pero estos se mantuvieron debido a su relevancia potencial para el análisis.

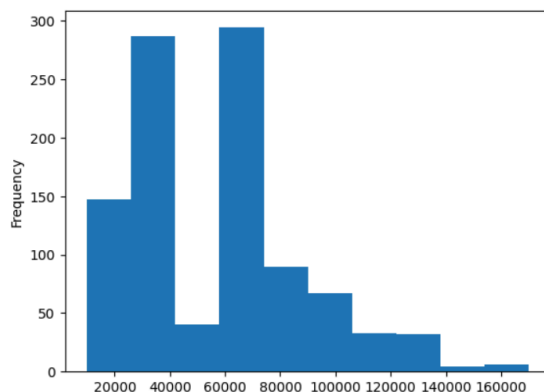
```
In [36]: #Estimación de Densidad Kernel (KDE) para la columna "Age"
df["Age"].plot.kde()
```

```
Out[36]: <Axes: ylabel='Density'>
```



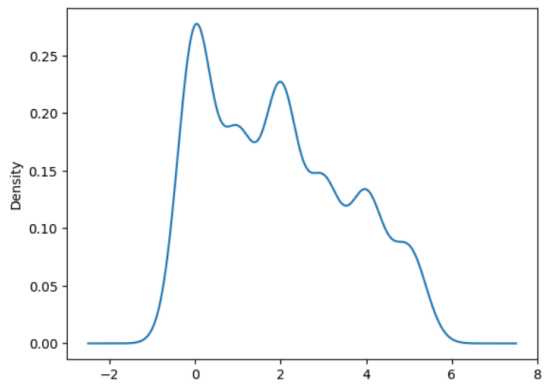
```
In [37]: # Histograma de la columna Income
df["Income"].plot.hist()
```

```
Out[37]: <Axes: ylabel='Frequency'>
```

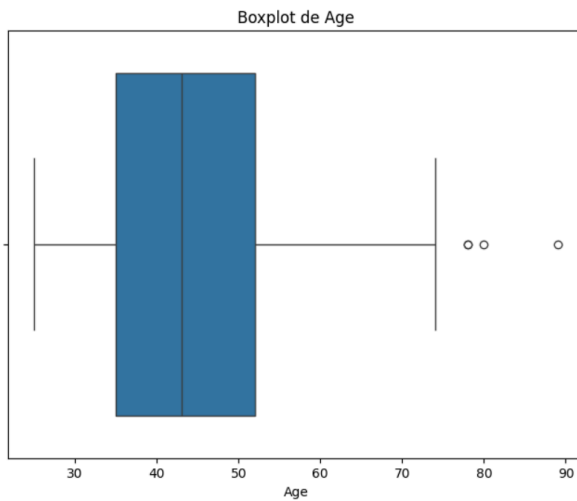


```
In [38]: #Estimación de Densidad Kernel (KDE) para la columna "Children"
df["Children"].plot.kde()
```

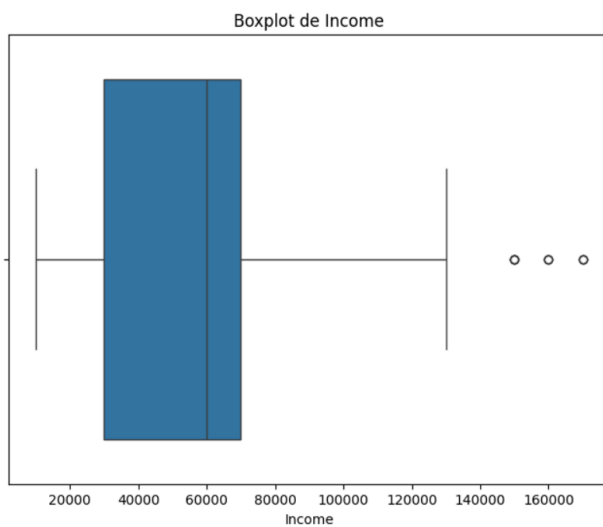
```
Out[38]: <Axes: ylabel='Density'>
```



```
In [39]: # Boxplot de la columna 'Age'
plt.figure(figsize=(8, 6))
sns.boxplot(x=df['Age'])
plt.title('Boxplot de Age')
plt.show()
```



```
In [40]: # Boxplot de la columna 'Income'
plt.figure(figsize=(8, 6))
sns.boxplot(x=df['Income'])
plt.title('Boxplot de Income')
plt.show()
```



5. Conclusiones

El proceso de limpieza mejoró significativamente la calidad del conjunto de datos, haciendo que sea más representativo y adecuado para el análisis. Se eliminaron errores de formato, se imputaron valores faltantes de manera adecuada y se gestionaron los valores atípicos. Con estos datos limpios, el análisis descriptivo y predictivo será más fiable y conducente a conclusiones más válidas.

6. Recomendaciones

Se recomienda automatizar el proceso de limpieza en futuras iteraciones, utilizando scripts o herramientas que permitan la verificación continua de la calidad de los datos. También sería útil implementar mecanismos de control en la recolección de datos para reducir la introducción de errores. El uso de técnicas avanzadas de detección de outliers o imputación de valores faltantes puede mejorar aún más la precisión del análisis en el futuro.

7. Referencias

The pandas development team. (2023). pandas: Python Data Analysis Library. pandas. <https://pandas.pydata.org/>

Waskom, M. (2023). Seaborn: Statistical Data Visualization. Seaborn. <https://seaborn.pydata.org/>

Hunter, J. D., Caswell, T. A., Droettboom, M., Firing, E., & Lee, A. (2023). Matplotlib: Python plotting — Matplotlib 3.7.0 documentation. Matplotlib. <https://matplotlib.org/stable/>

Python Software Foundation. (2023). The Python Language Reference. Python. <https://docs.python.org/3/>

McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 51–56. <https://doi.org/10.25080/Majora-92bfl922-00a>